

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Experiment No.: PBL Q.2 (CO-2)

Student Name: Kottana Roshini

Roll No.: A24126552263

1. TITLE

Node.js File Management Application using the File System (fs) Module

2. AIM

To develop a Node.js application that accepts a file name and content from the user and performs create/write, read, append and display operations using the File System module.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the Node.js File System (fs) module.
- To create and write a file using Node.js.
- To read the contents of a file.
- To append additional content to an existing file.
- To accept input from the user in the terminal and handle errors.

4. THEORY

- Node.js is a JavaScript runtime that allows JavaScript to run outside the browser, so it can access files and the operating system.
- fs is a built-in Node.js module used for file operations. It does not need to be installed.
- fs.writeFileSync(path, data) creates a file, or overwrites it if it already exists.
- fs.readFileSync(path, "utf8") reads the complete contents of a file as text.
- fs.appendFileSync(path, data) adds data at the end of an existing file.
- fs.existsSync(path) checks whether a file already exists.
- The Sync versions run step by step and wait for each operation to finish; asynchronous versions (fs.writeFile, etc.) use callbacks or promises.
- The readline module reads input typed by the user in the terminal, and the path module handles file paths safely.

The basic flow of the application is:

User Input -> Create/Write File -> Read File -> Append Content -> Read and Display Final Content

5. ALGORITHM / PROCEDURE

1. Create a new file named fileManager.js.
2. Import the fs, path and readline modules.
3. Create a readline interface to read input from the terminal.
4. Ask the user to enter a file name; stop with an error if it is empty.
5. Use path.basename() to remove folder paths and add .txt if there is no extension.
6. Ask the user to enter the file content.

7. Create/write the file using `fs.writeFileSync()`.
8. Read the file using `fs.readFileSync()` and display its contents.
9. Ask the user for additional content and append it using `fs.appendFileSync()`.
10. Read the file again and display the final contents.
11. Handle any errors using `try...catch` and close the `readline` interface.
12. Run the program using `node fileManager.js` and test with different inputs.

6. PROGRAM

`fileManager.js`

```
// fileManager.js
// Node.js File Management using the built-in File System (fs) module.
// Operations: create/write a file, read it, append content, display final text.

// Import the built-in modules (no installation needed)
const fs = require("fs");           // File System: create, read, append files
const path = require("path");        // Path: work safely with file names
const readline = require("readline"); // Readline: take input from the terminal

// Create an interface to read what the user types in the terminal
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Lets us read the typed lines one by one using "await"
const lines = rl[Symbol.asyncIterator]();

// Helper: show a question and wait for the user's answer
async function ask(question) {
  process.stdout.write(question);
  const { value } = await lines.next();
  return value === undefined ? "" : value;
}

// Helper: read a file and display its contents on the screen
function displayFile(filePath, heading) {
  // Read the whole file as text (utf8)
  const data = fs.readFileSync(filePath, "utf8");
  console.log("\n--- " + heading + " ---");
  console.log(data);
  console.log("-----");
}

async function main() {
  // try...catch handles any file error (for example, permission problems)
  try {
    // 1. Accept the file name and content from the user
    let fileName = (await ask("Enter file name: ")).trim();
    if (fileName === "") {
      // Validation: an empty name is not allowed
      console.log("Error: File name cannot be empty.");
      return;
    }
    fileName = path.basename(fileName); // avoid folder paths
    if (path.extname(fileName) === "") {
      fileName += ".txt"; // default extension
    }
  }
}
```

```

    }
    // Full path: the file is created in the same folder as this program
    const filePath = path.join(__dirname, fileName);
    const content = await ask("Enter file content: ");

    // 2. Create / write the file (an existing file is overwritten)
    const existed = fs.existsSync(filePath);
    fs.writeFileSync(filePath, content);
    const status = existed ? "Existing file overwritten: " : "File created: ";
    console.log("\n" + status + fileName);

    // 3. Read the contents
    displayFile(filePath, "File contents after writing");

    // 4. Append additional content at the end of the file
    const extra = await ask("Enter additional content to append: ");
    fs.appendFileSync(filePath, "\n" + extra);
    console.log("\nAdditional content appended successfully.");

    // 5. Display the final contents
    displayFile(filePath, "Final file contents");
  } catch (err) {
    // Runs if any operation above fails
    console.log("File operation failed: " + err.message);
  } finally {
    // Always close the readline interface so the program can exit
    rl.close();
  }
}

// Start the program
main();

```

7. CODE EXPLANATION

Code	Explanation
<code>require("fs")</code>	Imports the built-in File System module.
<code>readline.createInterface()</code>	Creates an interface to read user input from the terminal.
<code>ask(question)</code>	Displays a question and waits for the user's answer.
<code>path.basename()</code>	Keeps only the file name so the file is created in the project folder.
<code>fs.existsSync()</code>	Checks whether the file already exists.
<code>fs.writeFileSync()</code>	Creates the file and writes the content.
<code>fs.readFileSync()</code>	Reads the file contents.
<code>fs.appendFileSync()</code>	Adds the additional content at the end of the file.
<code>try ... catch ... finally</code>	Handles errors and always closes the readline interface.

8. TEST CASES AND EXECUTION DATA

The program was executed in the terminal for each test case and the actual results were recorded.

Test Case	Input	Expected Result	Actual Result	Status
TC-01	Name: notes Content: Node.js is a JavaScript runtime. Append: Express.js is a web framework.	File notes.txt created; final content has both lines	Message: File created: notes.txt Final: Node.js is a JavaScript runtime. Express.js is a web framework.	Pass
TC-02	Name: student.txt Content: Name: Roshini Append: Course: Full Stack Web Development	File student.txt created; final content has both lines	Message: File created: student.txt Final: Name: Roshini Course: Full Stack Web Development	Pass
TC-03	Name: notes Content: Fresh content replaces the old one. Append: Appended line after overwrite.	Existing notes.txt overwritten; appended line added	Message: Existing file overwritten: notes.txt Final: Fresh content replaces the old one. Appended line after overwrite.	Pass
TC-04	Name: (empty)	Error: file name cannot be empty; no file created	Message: Error: File name cannot be empty.	Pass

9. OUTPUT

Output 1: TC-01 - File created, read, appended and displayed

```
Terminal
$ node fileManager.js
Enter file name: notes
Enter file content: Node.js is a JavaScript runtime.

File created: notes.txt

--- File contents after writing ---
Node.js is a JavaScript runtime.
-----
Enter additional content to append: Express.js is a web framework.

Additional content appended successfully.

--- Final file contents ---
Node.js is a JavaScript runtime.
Express.js is a web framework.
-----
```

Output 2: TC-03 - Existing file overwritten, then appended

```
Terminal
$ node fileManager.js
Enter file name: notes
Enter file content: Fresh content replaces the old one.

Existing file overwritten: notes.txt

--- File contents after writing ---
Fresh content replaces the old one.
-----
Enter additional content to append: Appended line after overwrite.

Additional content appended successfully.

--- Final file contents ---
Fresh content replaces the old one.
Appended line after overwrite.
-----
```

Output 3: TC-04 - Empty file name shows an error

```
Terminal
$ node fileManager.js
Enter file name:
Error: File name cannot be empty.
```

10. RESULT

Thus, a Node.js file management application was successfully developed using the File System module. The program creates/writes a file, reads its contents, appends additional content and displays the final contents, and it was verified using multiple test cases.